Internship Report

AI-Assisted Automation Platform for Interface Control Document (ICD) Generation

Name: Kinthada Baladithya
Role: Engineer (Intern)
Internship Start Date: 15-12-2025
Internship End Date: 13-03-2026
Internship Manager:

Gaurav Rohilla
Digital SL
Cyient | Mobile: +91 9958900509
Office Location: Cyient Manikonda, Hyderabad, Telangana

Table of Contents

1. Introduction
2. Architecture Diagram
3. Tools and Technologies Used
4. Internship Responsibilities
5. Learning and Development
6. Challenges and Solutions
7. Conclusion

Introduction

During my internship at **Cyient**, I worked on designing and implementing an automated solution for generating **production-ready Interface Control Documents (ICDs)** used in large-scale enterprise systems. The main objective of the project was to reduce manual documentation effort while improving **consistency, traceability, and compliance with enterprise system requirements**.

The project involved developing an **AI-Assisted ICD Automation Platform**, which programmatically generates structured and styled ICD documents for **Concession Instance APIs** aligned with **NZ-REQ-1265 to NZ-REQ-1278** requirements.

The platform automatically generates structured documentation including:

- API interface definitions
- Request and response schemas
- Error handling models
- Interface lifecycle flows
- Requirement-level sequence diagrams
- Verification and interface mapping details

The generated outputs follow a **standard enterprise ICD format** and are exported in both **Word and PDF formats**,

ensuring consistent styling, section hierarchy, and formatting using a reference template. The system also integrates contextual information from **System Architecture Documents (SAD)**, **High-Level Design (HLD)** artifacts, and API specifications to ensure the generated documentation bodies **accurate, traceable, and aligned with enterprise standards**.

Architecture Diagram

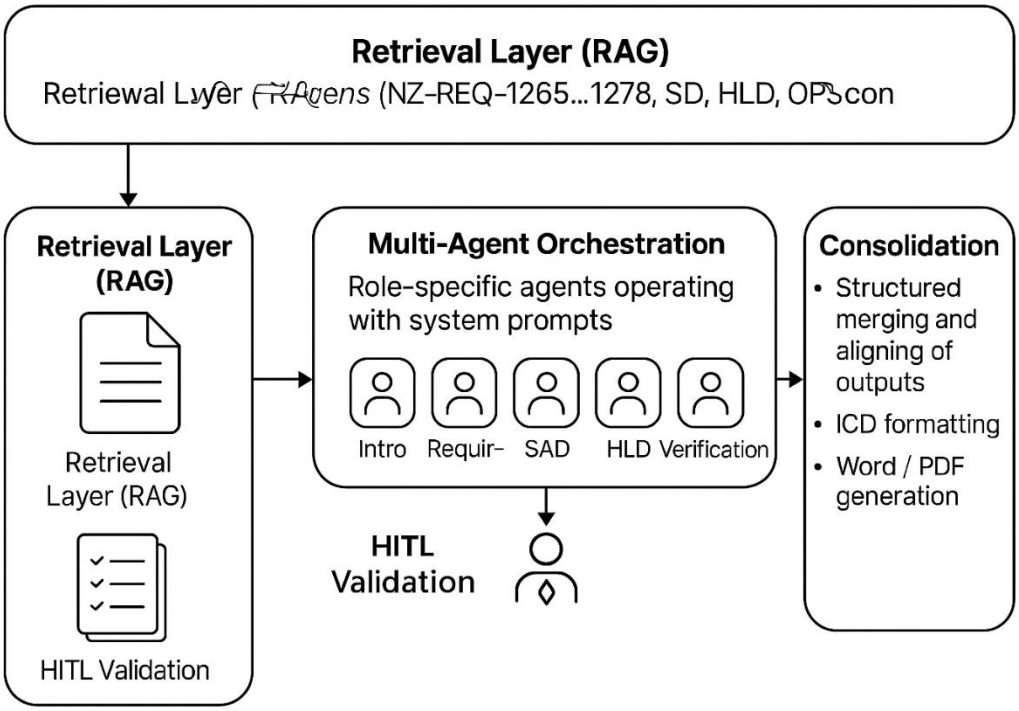

Figure : Architecture of the AI-Assisted ICD Automation Platform

The architecture of the system follows a Retrieval-Augmented Generation (RAG) based multi-agent pipeline

designed to generate enterprise-grade Interface Control Documents.

The architecture consists of three primary layers:-

1. Retrieval Layer (RAG): This layer retrieves contextual information from architecture and design artifacts such as **SAD, HLD, API specifications, and requirement documents.** Azure AI Search is used to perform vector and keyword-based retrieval, ensuring relevant context is supplied to the AI agents.

2. Multi-Agent Orchestration Layer: This layer coordinates **multiple AI agents** responsible for generating different sections of the ICD. Each agent operates with system prompts and role-specific instructions to produce structured outputs such **as introduction, requirement mapping, interface definitions, verification details, and traceability information.** Human-in-the-loop validation is included to ensure correctness.

3. Consolidation and Document Generation Layer: The consolidation agent merges the outputs generated by **different agents, aligns them with the ICD template, and formats the document into structured sections.** The final output is generated in Word and PDF formats, including **embedded diagrams, schema tables, and interface definitions.**

Validation mechanisms such **as traceability checks, schema verification, and diagram validation** are applied

to ensure that the generated ICD aligns with enterprise documentation standards.

Tools and Technologies Used

The project utilized a combination of **software engineering tools, AI technologies, and documentation frameworks** to automate ICD generation.

Programming & Frameworks

- Python
- python-docx
- ReportLab
- Matplotlib

Artificial Intelligence Technologies

- Generative AI
- Agentic AI
- Azure OpenAI
- Retrieval-Augmented Generation (RAG)

Cloud & Search Technologies

- Azure AI Search
- Vector Search
- Embeddings-based document retrieval

Documentation & Visualization

- Microsoft Word (DOCX templates)
- PDF document generation
- Mermaid-style sequence diagrams
- Architecture and flow diagrams

Data & Integration Sources

- System Architecture Documents (SAD)
- High-Level Design Documents (HLD)
- API specifications
- Requirement datasets (NZ-REQ-1265 to NZ-REQ-1278)

Internship Responsibilities

Throughout the internship, I was responsible for both **technical design and implementation tasks** related to ICD automation.

My key responsibilities included:

• Analyzing **NZ-REQ-1265 to NZ-REQ-1278** requirements and mapping them to API functionalities such as:

- Concession issuance
- Retrieval and update operations

- Suspension and resumption
- Revocation workflows
- Fare-media associations
- Bulk operations
- Audit tracking

• Designing and developing the **AI-Assisted ICD Automation Platform** capable of generating full enterprise ICD documentation automatically.

• Implementing functionality to read a **reference DOCX template**, extract section hierarchy, headings, styles, and formatting rules.

• Automating the generation of structured ICD sections including:

- Numbered headings
- API interface tables
- Request/response schema tables
- Consistent Table of Contents

• Integrating **Azure AI Search** to retrieve contextual information from architecture and design artifacts.

• Implementing **Generative AI and Agentic AI components** using Azure OpenAI to generate controlled technical documentation content.

• Designing **prompt-scoped AI agents** to ensure documentation generation remains restricted to the defined NZ-REQ scope.

• Defining **standardized API documentation structures**, including:

- Request payload schemas
- Response formats
- Header definitions
- Lifecycle attributes

• Implementing **enterprise-grade error models** covering standard HTTP response codes:

- 400 – Bad Request
- 401 – Unauthorized
- 403 – Forbidden
- 404 – Not Found
- 409 – Conflict
- 500 – Internal Server Error

• Demonstrating **optimistic concurrency control** using **ETag and If-Match headers** within API documentation examples.

• Generating **requirement-level sequence diagrams** programmatically and embedding them within the document.

• Ensuring final document quality through **format validation, export checks, and layout consistency** for both Word and

PDF outputs.

Learning and Development

This internship significantly enhanced both my **technical capabilities and professional engineering skills**.

Major learnings include:

• Advanced usage of **Python for enterprise automation systems**, emphasizing modular and scalable design.

• Hands-on experience with **python-docx** for generating structured Word documents with styling, headings, and tables.

• Practical exposure to **ReportLab** for generating professional PDF documents.

• Understanding the structure and governance of **enterprise API documentation**.

• Experience with **Azure AI Search** for retrieving architecture and design context using both keyword and vector search.

• Exposure to **Generative AI and Agentic AI systems** used for controlled technical documentation generation.

• Designing **AI prompt-scoped agents** that ensure generated outputs remain within strict documentation boundaries.

• Learning best practices for **API documentation standards**, including:

- Semantic versioning
- Change management
- Requirements traceability
- Interface verification planning

• Improved understanding of **system integration concepts**, including interface contracts, lifecycle management, and data exchange.

• Strengthened **analytical thinking, debugging, and technical decision-making** skills in a real enterprise development environment.

Challenges and Solutions

Challenge 1: Handling Large and Unsupported Images in Word Documents

Problem: Some architecture and flow images were too large or incompatible with Word formatting standards. **Solution:** Implemented an automated image re-encoding and downscaling utility that converts images to optimized formats and caps maximum dimensions while preserving readability.

Challenge 2: Requirement Gaps While Maintaining Strict Scope Control

Problem: Certain implementation details were missing in source artifacts, but the ICD still needed to remain complete.
Solution: Clearly labeled such elements as *"Inferred from standard enterprise practices"* and strictly constrained them to the Concession API domain to avoid scope leakage.

Challenge 3: Maintaining Visual Parity Between Word and PDF Outputs

Problem: Differences in layout engines caused inconsistencies in styles, margins, and colors.
Solution: Parsed the reference DOCX structure and explicitly applied the same hierarchy, spacing, and visual profile to both Word and PDF generation pipelines.

Challenge 4: Sequence Diagram Rendering Without External Dependencies

Problem: Standard diagram tools were not available in all environments.
Solution: Implemented a lightweight **Mermaid-style sequence diagram renderer** using Python and matplotlib as a fallback solution.

Conclusion

The internship project successfully delivered an **AI-Assisted ICD Automation Platform** capable of generating enterprise-grade Interface Control Documents with minimal manual intervention.

The platform significantly improves documentation efficiency while ensuring **accuracy, traceability, and compliance with enterprise standards**.

By integrating **Generative AI, Agentic AI, and retrieval-based knowledge systems**, the solution demonstrates how modern AI technologies can enhance enterprise documentation workflows.

This internship provided valuable exposure to **real-world enterprise systems, AI-assisted engineering practices, and API governance frameworks**.

Overall, my internship at **Cyient** was a highly enriching experience that strengthened my **technical skills, problem-solving abilities, and understanding of enterprise software development practices**.